

Where the MCP Python SDK lives in the code

The SDK is pinned in `requirements.txt:12` as `mcp==1.26.0` . Two SDK imports are used (`mcp_client.py:21-23`):

```
from mcp import ClientSession
from mcp.client.streamable_http import streamable_http_client
```

- `ClientSession` — high-level MCP session object. Speaks the JSON-RPC protocol; exposes `initialize()`, `list_tools()`, `call_tool()` .
- `streamable_http_client` — opens an HTTP-based bidirectional stream to the MCP server and yields `(read, write, get_session_id)` .

The MCP client in this project

Single file: `app/mcp_client.py` — wraps the SDK in a `MCPClient` class plus three module helpers (`init_mcp`, `get_mcp`, `close_mcp`) that manage a per-process singleton.

Key responsibilities (`mcp_client.py`):

Method	What it does
<code>connect()</code> (line 47)	Opens HTTP transport via <code>streamable_http_client</code> , builds a <code>ClientSession</code> , calls <code>session.initialize()</code> , then <code>list_tools()</code> and converts each tool to OpenAI "function" schema.
<code>call_tool(name, args)</code> (line 104)	Delegates to <code>session.call_tool(...)</code> — the actual MCP RPC.
<code>close()</code> (line 115)	<code>aclose()</code> on the <code>AsyncExitStack</code> — tears down session and HTTP transport in LIFO order.

`AsyncExitStack` is used so the transport + session live across `connect()` / `close()` instead of one `async` with `block`.

Files that participate in the MCP flow

File	Role
<code>app/mcp_client.py</code>	Owns the SDK objects. Defines <code>MCPClient</code> and the singleton helpers.
<code>app/main.py</code>	Lifecycle: <code>init_mcp(cfg.mcp_server_url)</code> at startup (line 57), <code>close_mcp()</code> at shutdown (line 64) — wired into FastAPI's lifespan hook.
<code>app/agent.py</code>	Consumer: <code>mcp = get_mcp()</code> (line 99) → reads <code>mcp.tool_schemas</code> to pass to the LLM, then calls <code>await mcp.call_tool(name, arguments=args)</code> (line 240) inside the agent loop. <code>_mcp_result_to_text()</code> (line 77) flattens <code>mcp_result.content</code> (list of <code>TextContent</code>) into a plain string for the LLM.
<code>app/odoo_config.py</code>	Provides <code>mcp_server_url</code> — pulled from Odoo's <code>ir.config_parameter</code> at startup.
<code>app/routers/ops.py</code>	Exposes <code>/health/mcp</code> for ops to check the connection.

Lifecycle in one picture

```
FastAPI startup -> init_mcp(url) -> MCPClient.connect()
                                |- streamable_http_client(url) <- SDK
                                |- ClientSession(read, write) <- SDK
                                |- session.initialize() <- SDK
                                +- session.list_tools() <- SDK

each /message -> agent.py: get_mcp().call_tool(name, args) <- SDK

FastAPI shutdown -> close_mcp() -> MCPClient.close() -> exit_stack.aclose()
```

So the SDK is touched in exactly one file (`mcp_client.py`); every other file talks to the SDK indirectly through that wrapper.